

UNIT 5: NEURAL NETWORKS, DEEP LEARNING & ADVANCED AI

Course Pattern: TE IT - 2024 Pattern (SPPU)	Academic Term: Semester V (2026)
Status: Publication-Grade Theory Guide	Course Code: PCC-302-IT

Syllabus Mapping & Unit Structure

The following table outlines the exam-oriented curriculum mapping and priority weightage for Unit 5 based on SPPU end-semester examination trends.

Syllabus Topic	In-Depth Exam Coverage	Priority / Weightage
ANN & Perceptron Learning	Biological vs Artificial neurons, McCulloch-Pitts model, Rosenblatt's Perceptron, Perceptron Learning Algorithm, XOR linear inseparability limitation.	High Priority (6-8 Marks Qs)
Activation Functions & MLP	Sigmoid, Tanh, ReLU, Leaky ReLU, Softmax, Multilayer Perceptron (MLP) architecture, Feedforward calculation, Weight initialization (Xavier, He).	High Priority (8-10 Marks Qs)
Error Backpropagation	Backpropagation algorithm derivation using Chain Rule, Gradient Descent weight updates, Loss functions (MSE, Cross-Entropy), Momentum parameter.	High Priority (8-10 Marks Qs)
Probabilistic Reasoning & BBN	Bayes' rule, Bayesian Belief Networks (BBN), Directed Acyclic Graphs (DAG), Conditional Probability Tables (CPTs), Markov Blanket, d-separation.	High Priority (7-9 Marks Qs)
Sampling & Markov Models	Prior Sampling, Rejection Sampling, Likelihood Weighting, Markov Chains, Hidden Markov Models (HMM), Three Fundamental Problems (Forward, Viterbi, Baum-Welch).	High Priority (8-10 Marks Qs)
Constraint Satisfaction (CSP)	Variables, Domains, Constraints, Map Coloring, Cryptarithmic, AC-3 Arc Consistency, MRV, Degree, LCV heuristics, Branch & Bound 0/1 Knapsack.	High Priority (8-10 Marks Qs)
Fuzzy Logic, NLP & LLMs	Fuzzy vs Crisp sets, Membership functions, Fuzzy operations, Mamdani FIS, NLP pipeline (TF-IDF, Embeddings), Transformer Self-Attention, Tokenization & LLMs.	High Priority (8-10 Marks Qs)

Artificial Neural Networks & Deep Learning Foundations

Biological Inspiration vs. Artificial Neuron Architecture

Artificial Neural Networks (ANNs) are massively parallel distributed computational models inspired by the structural and functional biological neural networks in the human brain.

Biological Neuron

- **Dendrites:** Receive electrochemical signals.
- **Soma (Cell Body):** Sums incoming signals.
- **Axon:** Transmits action potential spike.
- **Synaptic Terminals:** Modulates signal strength.

Artificial Neuron (Perceptron)

- **Input Signals** ($x_1, \dots, x_d$): Feature attributes.
- **Weights** ($w_1, \dots, w_d$) & **Bias** (b): Synaptic strengths.
- **Net Summation** ($z = w^T x + b$): Soma integrator.
- **Activation Function** ($f(z)$): Action threshold.

Figure 5.1: Biological Neuron Anatomy vs Artificial Neuron Computational Model

Foundational Neuron Models: McCulloch-Pitts & Rosenblatt's Perceptron

1. McCulloch-Pitts (M-P) Neuron Model (1943):

- Earliest simplified mathematical abstraction of biological neurons.
- Accepts binary boolean inputs $x_i \in \{0, 1\}$ with two distinct input types: **Excitatory** (+1) and **Inhibitory** (0).
- **Firing Rule:** If any inhibitory input is active, the neuron is inhibited ($y = 0$). Otherwise, it fires ($y = 1$) if the sum of excitatory inputs equals or exceeds a fixed preset threshold θ :

$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^d x_i \geq \theta \text{ and no inhibitory input active} \\ 0 & \text{otherwise} \end{cases}$$

- **Limitation:** Weights are fixed and equal; cannot learn autonomously from data.

2. Rosenblatt's Perceptron (1958):

- Introduces learnable real-valued synaptic connection weights $w \in \mathbb{R}^d$ and an adjustable bias $b = -\theta$:

$$z = \sum_{i=1}^d w_i x_i + b = w^T x + b$$

$$\hat{y} = \text{step}(z) = \begin{cases} 1 & \text{if } w^T x + b \geq 0 \\ 0 & \text{if } w^T x + b < 0 \end{cases}$$

3. Perceptron Learning Algorithm & Weight Update Rule: For each training sample $(x^{(i)}, y^{(i)})$:

- Compute model prediction $\hat{y}^{(i)} = \text{step}(w^T x^{(i)} + b)$.
- Calculate error: $e^{(i)} = y^{(i)} - \hat{y}^{(i)}$.
- Update weights and bias proportionally to learning rate $\eta \in (0, 1]$:

$$w_j := w_j + \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)}$$

$$b := b + \eta(y^{(i)} - \hat{y}^{(i)})$$

4. Perceptron Convergence Theorem & XOR Limitation (Minsky & Papert, 1969):

- **Convergence Theorem:** If the training dataset is **linearly separable**, the perceptron learning algorithm is mathematically guaranteed to converge to a separating hyperplane in a finite number of iterations.
- **The XOR Limitation:** A single-layer perceptron can only compute linearly separable boolean functions (AND, OR, NAND, NOR). It is mathematically impossible for a single hyperplane to separate the non-linear XOR (Exclusive-OR) / XNOR function. Overcoming this limitation required **Multilayer Perceptrons (MLPs)** equipped with non-linear activation functions.

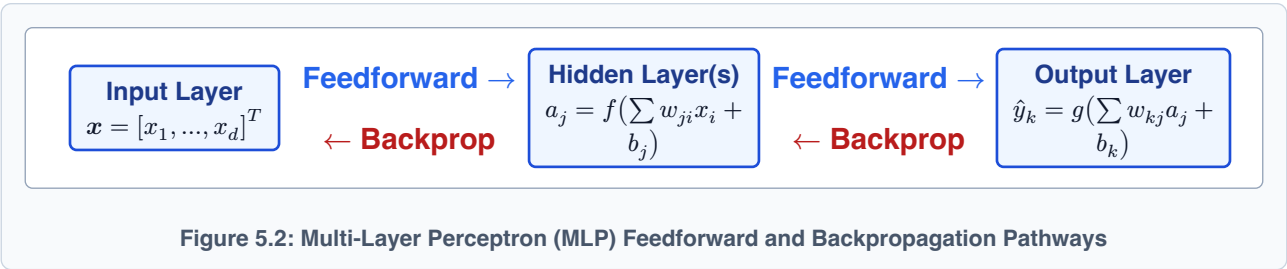
Comprehensive Analysis of Neural Activation Functions

Activation functions introduce non-linearity into neural networks, enabling them to approximate arbitrary complex mathematical mappings (Universal Approximation Theorem).

Function	Mathematical mula	For-	Output Range	Derivative ($f'(z)$)	Exam Properties & Pit-falls
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$		$(0, 1)$	$\sigma(z)(1 - \sigma(z))$	Smooth, probabilistic output. Prone to Vanishing Gradient at extreme $ z $; not zero-centered.
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$		$(-1, +1)$	$1 - \tanh^2(z)$	Zero-centered outputs. Still suffers from vanishing gradients for large $ z $.
ReLU	$\text{ReLU}(z) = \max(0, z)$		$[0, +\infty)$	$\begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$	Computationally efficient, prevents vanishing gradient for $z > 0$. Prone to Dying ReLU problem.
Leaky ReLU	$\max(\alpha z, z), \quad \alpha \approx 0.01$		$(-\infty, +\infty)$	$\begin{cases} 1 & \text{if } z > 0 \\ \alpha & \text{if } z \leq 0 \end{cases}$	Prevents dying ReLU by allowing a small constant non-zero gradient for $z < 0$.
Softmax	$\sigma(z)_i = \frac{e^{z_i}}{\sum_{k=1}^K e^{z_k}}$		$(0, 1), \sum = 1$	$\sigma_i(\delta_{ij} - \sigma_j)$	Standard output layer for multiclass classification; normalizes logits into class probability distribution.

Multilayer Perceptron (MLP) & The Error Backpropagation Algorithm

A Multilayer Perceptron is a fully connected feedforward artificial neural network consisting of at least three layers: an **Input Layer**, one or more **Hidden Layers**, and an **Output Layer**.



1. Forward Propagation Pass:
- Net input to hidden neuron j : $z_j = \sum_i w_{ji}x_i + b_j$.

• Activation of hidden neuron j : $a_j = f(z_j)$.

• Net input to output neuron k : $z_k = \sum_j w_{kj}a_j + b_k$.

• Final predicted output: $\hat{y}_k = g(z_k)$.
2. Loss Function (Mean Squared Error or Cross-Entropy):
- $$E = \frac{1}{2} \sum_{k=1}^K (y_k - \hat{y}_k)^2$$
3. Backward Propagation Pass (Derivation via Chain Rule):
- Output Layer Weight Gradients:

$$\frac{\partial E}{\partial w_{kj}} = \frac{\partial E}{\partial \hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{kj}} = -(y_k - \hat{y}_k) \cdot g'(z_k) \cdot a_j = \delta_k \cdot a_j$$

where the output error term is $\delta_k = -(y_k - \hat{y}_k)g'(z_k)$.

• **Hidden Layer Weight Gradients:**

$$\frac{\partial E}{\partial w_{ji}} = \left(\sum_k \frac{\partial E}{\partial z_k} \cdot \frac{\partial z_k}{\partial a_j} \right) \cdot \frac{\partial a_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ji}} = \left(\sum_k \delta_k w_{kj} \right) \cdot f'(z_j) \cdot x_i = \delta_j \cdot x_i$$

where the hidden error term is $\delta_j = \left(\sum_k \delta_k w_{kj} \right) f'(z_j)$.

4. **Weight and Bias Updates:**

$$w_{kj} := w_{kj} - \eta \frac{\partial E}{\partial w_{kj}} = w_{kj} - \eta \delta_k a_j$$

$$w_{ji} := w_{ji} - \eta \frac{\partial E}{\partial w_{ji}} = w_{ji} - \eta \delta_j x_i$$

5. **Weight Initialization Strategies:**

- Initializing all weights to zero causes **symmetry breaking failure** (all hidden neurons compute identical gradients).
- **Xavier (Glorot) Initialization:** $\text{Var}(W) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$ (optimal for Sigmoid/Tanh).
- **He Initialization:** $\text{Var}(W) = \frac{2}{n_{\text{in}}}$ (mandatory for ReLU/Leaky ReLU).

Source: Russell & Norvig, Ch 21, p. 750-775 | Source: Marsland, Ch 3-4, p. 45-98 | Source: Alpaydin, Ch 11, p. 237-270

Probabilistic Reasoning, Bayesian Networks & Markov Models

Bayesian Belief Networks (BBN)

A Bayesian Belief Network is a probabilistic graphical model that represents a set of random variables and their conditional dependencies via a **Directed Acyclic Graph (DAG)**.

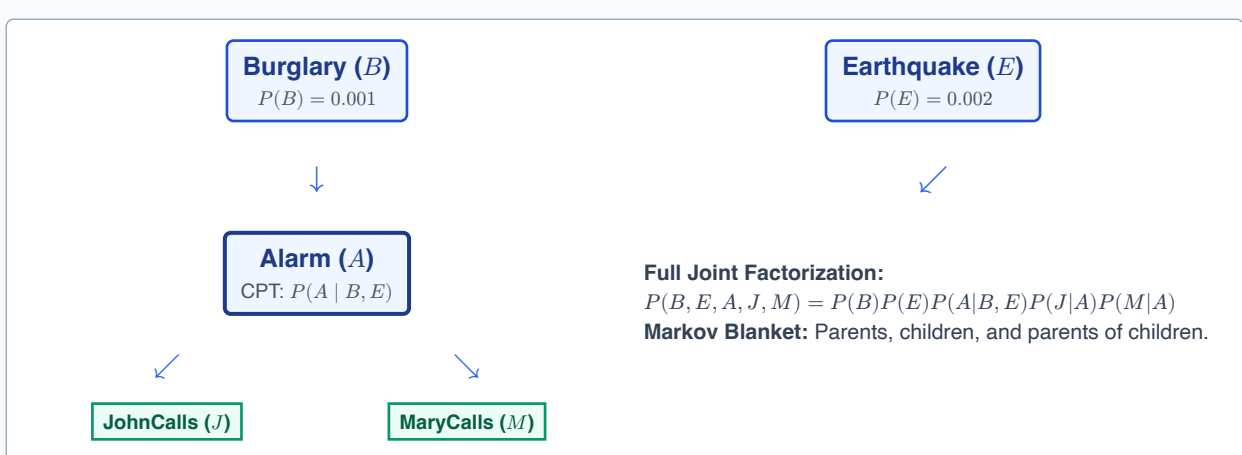


Figure 5.3: Bayesian Belief Network Topology and Local Conditional Independence

1. **Topological Semantics & Joint Probability Factorization:**

- Each node represents a random variable X_i .
- Each directed edge $X_i \rightarrow X_j$ represents direct causal influence (X_i is a parent of X_j).
- Each node contains a **Conditional Probability Table (CPT)** specifying $P(X_i | \text{Parents}(X_i))$.

- **Chain Rule of Bayesian Networks:** The complete joint probability distribution over all n variables factors into the product of local conditional distributions:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$$

2. **Conditional Independence & The Markov Blanket:**
- A node X is conditionally independent of all non-descendants given its parents.
 - **Markov Blanket:** Consists of a node's **parents**, **children**, and **children's other parents**. A node is conditionally independent of all other nodes in the entire network given its Markov Blanket.

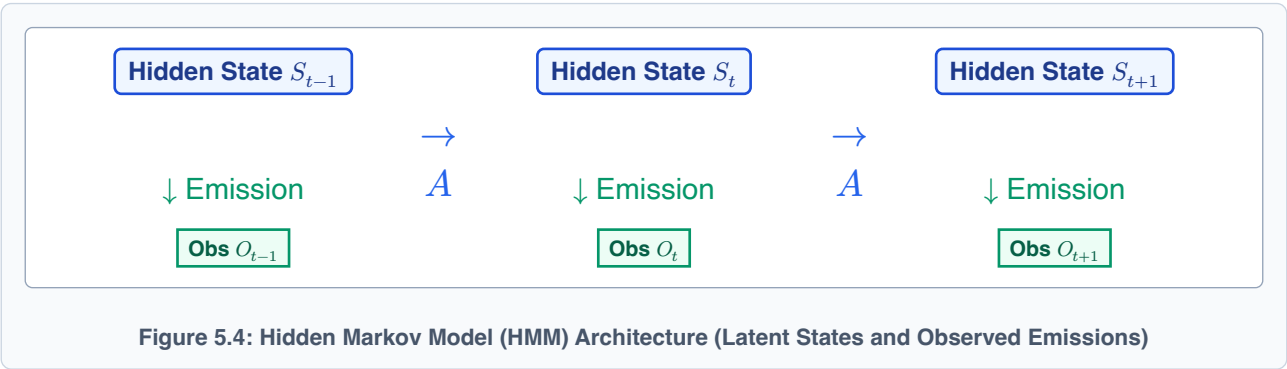
Approximate Inference via Stochastic Sampling Algorithms

Exact inference in general Bayesian networks is NP-hard. Approximate inference uses Monte Carlo stochastic simulation to generate empirical samples from the joint distribution.

Sampling Method	Generation Mechanism	Key Advantages	Primary Limitations
Prior Sampling	Samples variables in topological order according to prior/conditional CPTs.	Simple, fast, generates unconditional distribution.	Cannot condition on observed evidence without filtering.
Rejection Sampling	Generates prior samples; rejects any sample inconsistent with evidence e .	Produces exact posterior distribution $P(X \mid e)$.	Fraction of accepted samples drops exponentially with evidence size.
Likelihood Weighting	Fixes evidence variables to observed values; weights each sample by likelihood $w = \prod P(e_i \mid \text{Parents})$.	Zero wasted samples; all generated samples are retained.	Weight collapse occurs if evidence variables occur late in graph.
Gibbs Sampling (MCMC)	Markov Chain Monte Carlo: iteratively resamples non-evidence variables conditioned on their Markov Blankets.	Scalable to large networks with numerous evidence variables.	Samples are temporally correlated; requires initial burn-in period.

Markov Models & Hidden Markov Models (HMM)

A Markov Model represents dynamic systems where state changes over discrete time steps t .



1. **First-Order Markov Property & Stationary Assumption:**
- **Markov Assumption:** Future state depends solely on the immediate current state:
$$P(S_{t+1} \mid S_t, S_{t-1}, \dots, S_1) = P(S_{t+1} \mid S_t)$$
 - **Stationary Transition Probabilities:** $P(S_{t+1} = j \mid S_t = i) = A_{ij}$ is constant over time.

2. **Hidden Markov Model (HMM) Formal Definition:** An HMM is a bivariate stochastic process where the underlying state sequence $S_1, S_2, \dots$ is hidden (latent), but emits observable tokens $O_1, O_2, \dots$:
 - **Hidden State Space:** $S = \{s_1, s_2, \dots, s_N\}$.
 - **Observation Vocabulary:** $V = \{v_1, v_2, \dots, v_M\}$.
 - **State Transition Probability Matrix (A):** $A_{ij} = P(S_{t+1} = s_j \mid S_t = s_i)$.
 - **Observation Emission Probability Matrix (B):** $B_{jk} = P(O_t = v_k \mid S_t = s_j)$.
 - **Initial State Probability Distribution (π):** $\pi_i = P(S_1 = s_i)$.
 - Full HMM Model Parameter Set: $\lambda = (A, B, \pi)$.
3. **The Three Fundamental Canonical Problems of HMM:**
 - **Problem 1 (Evaluation / Likelihood):** Given model λ and observation sequence $O = (O_1, \dots, O_T)$, compute probability $P(O \mid \lambda)$. Solved via the **Forward-Backward Algorithm** ($\mathcal{O}(N^2T)$).
 - **Problem 2 (Decoding):** Given model λ and observations O , find the single most likely hidden state sequence $S^* = (S_1^*, \dots, S_T^*)$. Solved dynamically via the **Viterbi Algorithm** ($\mathcal{O}(N^2T)$).
 - **Problem 3 (Learning / Parameter Estimation):** Given observation sequences O , adjust parameters $\lambda = (A, B, \pi)$ to maximize $P(O \mid \lambda)$. Solved via the **Baum-Welch Algorithm (Expectation-Maximization - EM)**.

Source: Russell & Norvig, Ch 13-14, p. 432-515 | Source: Marsland, Ch 16, p. 322-350 | Source: Alpaydin, Ch 14-15, p. 341-395

Constraint Satisfaction Problems & Evolutionary Search

Formal Definition of Constraint Satisfaction Problems (CSP)

A Constraint Satisfaction Problem represents a problem state not as an opaque black-box, but as a structured set of variables with discrete constraint relationships.

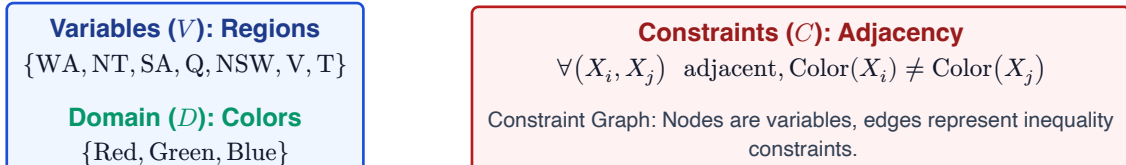


Figure 5.5: Map Coloring Constraint Satisfaction Graph and Constraint Propagation

1. **Formal Triplet Definition:** A CSP is defined as a tuple $\langle V, D, C \rangle$:
 - **Variables (V):** Set of n variables $V = \{X_1, X_2, \dots, X_n\}$.
 - **Domains (D):** Set of domains $D = \{D_1, D_2, \dots, D_n\}$, where each D_i lists allowed values for X_i .
 - **Constraints (C):** Set of constraint relations $C = \{C_1, C_2, \dots, C_m\}$, where each $C_i = \langle \text{scope}, \text{relation} \rangle$.
2. **Classic CSP Applications:**
 - **Map Coloring:** Color geographic territories using k colors such that no adjacent regions share the same color.
 - **Cryptarithmic Puzzles:** Assign unique digits $\{0, \dots, 9\}$ to letters (e.g., SEND + MORE = MONEY).
 - **N-Queens Problem:** Place N non-attacking queens on an $N \times N$ chessboard.

Constraint Propagation & The AC-3 Arc Consistency Algorithm

Constraint propagation interleaves domain reduction with search, pruning illegal domain values before branching.

1. **Arc Consistency (2-Consistency):** A directed arc (X_i, X_j) is **arc-consistent** if for every value $x \in D_i$, there exists some value $y \in D_j$ that satisfies the binary constraint between X_i and X_j .
2. **The AC-3 Algorithm:**
 - Initialize queue Q with all directed arcs in the CSP graph.

- While Q is non-empty:
 - Pop arc (X_i, X_j) from Q .
 - If Remove-Inconsistent-Values(X_i, X_j) removes any value from D_i :
 - If D_i becomes empty, return **Failure** (no solution exists).
 - Insert all neighbor arcs (X_k, X_i) where $X_k \neq X_j$ back into Q .
- Time complexity: $\mathcal{O}(cd^3)$ where c is number of binary constraints and d is maximum domain size.

Backtracking Search Heuristics

Backtracking Search is depth-first search that chooses values for one variable at a time and backtracks when a variable has no legal values.

1. **Minimum Remaining Values (MRV / “Fail-First” Heuristic):**
 - Choose the variable with the fewest legal values remaining in its domain.
2. **Degree Heuristic:**
 - Tie-breaker for MRV: Choose the variable with the most constraints on remaining unassigned variables.
3. **Least Constraining Value (LCV / “Fail-Last” Heuristic):**
 - Given a variable, choose the value that rules out the fewest choices for neighboring variables in the constraint graph.
4. **Forward Checking:** Whenever variable X is assigned, delete all inconsistent values from the domains of neighboring unassigned variables.

Branch and Bound Algorithm & 0/1 Knapsack Problem

Branch and Bound is an exact combinatorial optimization paradigm that prunes state-space subtrees whose optimistic upper bounds are lower than the best known feasible solution.

1. **Algorithm Mechanics:**
 - Maintain global best lower bound (LB) on the optimal solution.
 - Expand active node u ; calculate upper bound $UB(u)$ using greedy continuous relaxation.
 - If $UB(u) \leq LB$, **prune node u immediately** (it cannot contain a better solution).
2. **Application to 0/1 Knapsack Problem:**
 - Sort items in descending order of profit-to-weight ratio $\frac{p_i}{w_i}$.
 - Construct binary state tree (Include item i vs. Exclude item i).
 - Compute upper bound at node u using Fractional Knapsack relaxation:

$$UB = \text{current_profit} + \sum_{i \text{ fit}} p_i + (W_{\text{remaining}}) \cdot \left(\frac{p_{k+1}}{w_{k+1}} \right)$$

Genetic Algorithms (Evolutionary Optimization)

Genetic Algorithms are stochastic global search meta-heuristics inspired by Darwinian natural selection.

1. **Chromosome Encoding:** Candidate solutions represented as bitstrings, permutations, or real-valued vectors.
2. **Fitness Function:** Quantitative metric assessing survival quality of individual chromosome.
3. **Genetic Operators:**
 - **Selection:** Chooses parent individuals biased by fitness (**Roulette Wheel Selection** $P_i = \frac{f_i}{\sum f_j}$, **Tournament Selection**).
 - **Crossover (Recombination):** Exchanges genetic substrings between parent pairs at single or multiple crossover points.
 - **Mutation:** Randomly flips bits with small probability $p_m \approx 0.01$ to maintain population diversity and prevent premature convergence.

Source: Russell & Norvig, Ch 5, p. 161-190 | Source: Marsland, Ch 10, p. 209-225

Advanced AI: Fuzzy Logic, NLP Pipeline & Modern Transformers

Fuzzy Logic Systems

Fuzzy Logic extends classical Boolean logic to accommodate partial truth values between 0 (completely false) and 1 (completely true), modeling real-world linguistic vagueness.

Parameter	Classical (Crisp) Sets	Fuzzy Sets
Membership Valuation	Binary strict: $\mu_{A(x)} \in \{0, 1\}$ (Member or Non-member).	Continuous continuum: $\mu_{A(x)} \in [0, 1]$ (Degree of truth).
Boundary Nature	Crisp, sharp, unambiguous step threshold.	Gradual, smooth, overlapping transitions.
Core Operators	Boolean Algebra (AND, OR, NOT).	Zadeh Operators ($\min, \max, 1 - \mu_A$).
Excluded Middle Law	Holds strictly ($A \cup \bar{A} = U, A \cap \bar{A} = \emptyset$).	Violated ($A \cup \bar{A} \neq U, A \cap \bar{A} \neq \emptyset$).

1. Fuzzy Set Operations (Zadeh Triplet):
- **Fuzzy Union ($A \cup B$):** $\mu_{A \cup B}(x) = \max(\mu_{A(x)}, \mu_{B(x)})$
 - **Fuzzy Intersection ($A \cap B$):** $\mu_{A \cap B}(x) = \min(\mu_{A(x)}, \mu_{B(x)})$
 - **Fuzzy Complement ($\bar{A}$):** $\mu_{\bar{A}}(x) = 1 - \mu_{A(x)}$
2. Fuzzy Inference System (FIS - Mamdani Architecture):
- **Fuzzification:** Transforms crisp numeric input into fuzzy linguistic membership degrees.
 - **Rule Base:** Collection of IF-THEN linguistic rules (e.g., **IF** Temperature is High **AND** Humidity is High **THEN** Fan Speed is Fast).
 - **Aggregation:** Combines output fuzzy sets across all active rules using max operator.
 - **Defuzzification (Centroid Method):** Converts aggregate fuzzy area into crisp control output:

$$z^* = \frac{\int z \cdot \mu_{C(z)} dz}{\int \mu_{C(z)} dz} \quad \text{or} \quad z^* = \frac{\sum z_i \cdot \mu_{C(z_i)}}{\sum \mu_{C(z_i)}}$$

Natural Language Processing (NLP) Pipeline & Text Embeddings

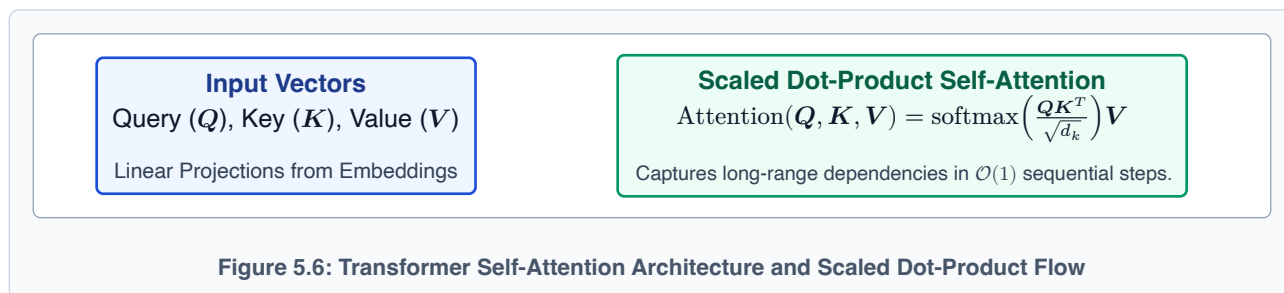
NLP bridges human natural linguistic communication and machine mathematical processing through multi-stage computational pipelines:

1. Preprocessing Pipeline:
- **Tokenization:** Segmenting raw text corpus into discrete lexical tokens (words or subwords via Byte-Pair Encoding - BPE).
 - **Normalization:** Case folding, punctuation removal, and stop-word filtering (removing non-informative words like **is, the, at**).
 - **Stemming vs. Lemmatization:**
 - **Stemming:** Crude heuristic rule-based suffix stripping (Porter Stemmer; converts **running** to **run**, **studies** to **studi**).
 - **Lemmatization:** Full morphological vocabulary analysis using WordNet to reduce word to its valid base dictionary lemma (**better** to **good**).
2. Vectorization & Embeddings:
- **Term Frequency-Inverse Document Frequency (TF-IDF):**

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \cdot \ln\left(\frac{N}{|\{d \in D \mid t \in d\}|}\right)$$

- Balances local term occurrence frequency against corpus-wide inverse rarity.
- **Dense Word Embeddings (Word2Vec / GloVe):** Maps semantic words into dense, continuous vector spaces $\mathbb{R}^d$ ($d \approx 300$) where cosine distance captures semantic similarity and linear arithmetic encodes relations ($v_{\text{King}} - v_{\text{Man}} + v_{\text{Woman}} \approx v_{\text{Queen}}$).

Large Language Models (LLMs) & Transformer Foundations



1. **The Self-Attention Mechanism (Vaswani et al., 2017):** Computes dynamic contextual relevance between every pair of tokens in a sequence simultaneously:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- $Q \in \mathbb{R}^{n \times d_k}$ (Query Matrix), $K \in \mathbb{R}^{n \times d_k}$ (Key Matrix), $V \in \mathbb{R}^{n \times d_v}$ (Value Matrix).
 - Scaling factor $\frac{1}{\sqrt{d_k}}$ prevents dot-product values from exploding into vanishing softmax gradient regions.
2. **Multi-Head Attention:** Runs h self-attention mechanisms in parallel across projected sub-spaces, allowing the model to jointly attend to information from different representation aspects.
 3. **Tokens vs. Model Parameters in Modern LLMs:**
 - **Tokens:** Basic atomic units of text processed by LLM subword tokenizers (1000 words $\approx$ 1333 tokens).
 - **Parameters:** Learnable synaptic weights in the transformer matrices. For instance, **LLaMA-3 8B** contains 8 billion parameters, while **LLaMA-3 70B** incorporates 70 billion parameters, requiring higher compute and quantization for inference.
 - **Fine-Tuning vs. Retrieval-Augmented Generation (RAG):** Fine-tuning updates internal model parameter weights; RAG dynamically retrieves authoritative external vector context into the prompt window without modifying weights.

Source: Russell & Norvig, Ch 21-23, p. 770-865 | Source: Marsland, Ch 4, p. 71-105